# BOROKO BOOKINGS

Technical Architecture & Capability Document

A deep-dive into the design, engineering, and operational scope of a full-stack hospitality management platform

Version 1.2.6 • April 2026 • Confidential

# Table of Contents

**1. Executive Overview**
Project goals, current state, and strategic positioning

**2. System Architecture**
Electron + React + Supabase hybrid model

**3. Technology Stack**
Libraries, build tools, and infrastructure

**4. Database Design**
Schema, relationships, and design principles

**5. Supabase RPC Functions**
All 50+ server-side functions explained

**6. Security Architecture**
RLS, session management, role capabilities

**7. Authentication System**
Desktop and PWA auth contracts

**8. Booking & Payment Engine**
Core financial operations and safety guarantees

**9. Point of Sale (POS) System**
Order workflow, outlets, folio integration

**10. Inventory & Room Supplies**
Stock tracking, purchases, and movement audit

**11. Reporting & Analytics**
All report types and dashboard snapshot

**12. Offline-First Synchronization**
SQLite cache, sync queue, and conflict resolution

**13. Feature Flags & Subscriptions**
Plan entitlements and per-lodge overrides

**14. Manager PWA**
Mobile app architecture, pages, and limitations

**15. Push Notifications & Broadcasts**
Real-time messaging infrastructure

**16. Email Integration**
SMTP setup, templates, and auto-send rules

**17. Data Import & Export**
Bulk operations and audit trails

**18. Performance & Build**
Code splitting, lazy loading, and release pipeline

**19. Known Issues & Roadmap**
Outstanding gaps and planned improvements

### 20. Conclusion

Architectural philosophy and key takeaways

# 1. Executive Overview

Boroko Bookings is a production-grade hospitality management platform engineered for lodges, resorts, guest houses, and restaurants operating in environments where reliable internet cannot be guaranteed. It is a **hybrid desktop/web application** that unifies bookings, guest management, payments, point of sale, inventory, housekeeping, maintenance, finance, and executive reporting into a single system accessible from both a native Windows desktop app and a mobile-first Progressive Web App (PWA).

## 1.1 Core Value Propositions

- **Offline-first reliability:** Operations continue even with no internet. All actions are queued and replayed when connectivity returns.
- **Financial integrity:** Every payment, refund, and charge is routed through server-side RPC functions that enforce atomicity and prevent double-dipping.
- **Multi-role access control:** 80+ granular capabilities across 8 predefined roles ensure staff see and do only what they are authorised for.
- **Multi-property support:** A single installation can manage multiple lodges from one login via lodge-scoped Row-Level Security (RLS).
- **Real-time manager oversight:** The companion PWA lets managers review live KPIs, bookings, and receive push notifications from any device.
- **Subscription-gated features:** Starter, Standard, and Pro plans unlock different capabilities, enforced both client-side and server-side.

## 1.2 Product at a Glance

| | |
|---|---|
| **Current Version** | 1.2.6 |
| **Primary Platform** | Electron 29 desktop application (Windows) |
| **Secondary Platform** | Manager PWA (React 19, hosted on Netlify) |
| **Backend** | Supabase (PostgreSQL 15, Edge Functions, Storage) |
| **Offline Storage** | SQLite via better-sqlite3 (per-lodge local database) |
| **Target Market** | Lodges, resorts, guest houses, and F&B; outlets in Africa |
| **Licensing** | SaaS subscription — Starter / Standard / Pro |
| **Auto-update** | GitHub Releases via electron-updater |

## 1.3 Modules Shipped

| Module | Starter | Standard | Pro |
|---|---|---|---|
| Bookings & Invoices | Yes | Yes | Yes |
| Guest Management | Yes | Yes | Yes |

| | | | |
|---|---|---|---|
| Room Setup & Rates | Yes | Yes | Yes |
| Housekeeping & Maintenance | Yes | Yes | Yes |
| Quotations | Yes | Yes | Yes |
| Reports & Analytics | No | Yes | Yes |
| Expense Tracking | No | Yes | Yes |
| Staff Management | No | Yes | Yes |
| Night Audit | No | Yes | Yes |
| Conference Bookings | No | Yes | Yes |
| Bulk Import | No | Yes | Yes |
| Point of Sale (POS) | No | No | Yes |
| Inventory Management | No | No | Yes |
| Room Supplies Tracking | No | No | Yes |
| Manager PWA | No | No | Yes |
| Push Notifications | No | No | Yes |

# 2. System Architecture

Boroko Bookings is built on a three-tier architecture: a native Electron desktop application, a cloud Supabase backend, and a mobile PWA. Each tier has clearly defined responsibilities and communicates through well-defined interfaces.

## 2.1 Architectural Overview

```
# High-Level Architecture
┌──────────────────────────────────────────────────────────────────────┐
│ DESKTOP (Electron 29) │
│ │
│ ┌──────────────────────────┐ ┌──────────────────────────────────┐ │
│ │ Renderer Process │ │ Main Process │ │
│ │ React 18 + Router │IPC│ database.js (13 000 lines) │ │
│ │ TailwindCSS UI │ │ emailNotifications.js │ │
│ │ Context API state │ │ Electron IPC handlers │ │
│ └──────────────────────────┘ └──────────────────────────────────┘ │
│ │ │ │
│ ┌──────────────▼──────────────────┐ │
│ │ Local SQLite (per lodge) │ │
│ │ Read-through cache │ │
│ │ Sync queue + failed queue │ │
│ └─────────────────────────────────┘ │
└──────────────────────────────────────────────────────────────────────┘
 HTTPS / Supabase SDK
┌──────────────────────────────────────▼──────────────────────────────┐
│ SUPABASE (PostgreSQL 15) │
│ 50+ RPC functions (SECURITY DEFINER) | Row-Level Security │
│ Edge Functions: send-push, send-booking-confirmation │
│ app_sessions table | push_subscriptions table │
└──────────────────────────────────▲──────────────────────────────────┘
 x-boroko-session header
┌──────────────────────────────────────────────────────────────────────┐
│ MANAGER PWA (React 19, Netlify) │
│ Direct Supabase SDK calls — no database.js dependency │
│ Service Worker | Push Notifications | 12-hour sessions │
└──────────────────────────────────────────────────────────────────────┘
```

## 2.2 Two Execution Paths

The most important architectural distinction in Boroko Bookings is that **the desktop app and the PWA take completely separate paths to reach the database**. Understanding this is critical for maintenance and debugging.

| | |
|---|---|
| **Desktop path** | UI → IPC call → database.js → Supabase RPC + SQLite write |
| **PWA path** | UI → lib/api.js → Supabase RPC (direct, no database.js) |
| **Convergence point** | Both MUST call the same Supabase RPC functions for data integrity |
| **Risk area** | Any mutation added to database.js must also be reflected in the PWA API layer |

## 2.3 IPC Bridge (Preload Script)

The Electron preload script (**src/preload/index.js**) acts as a secure bridge between the sandboxed renderer process and the privileged main process. It exposes a structured window.api object with namespaced handlers:

```
window.api = {
bookings: { getAll, create, update, cancel, ... },
payments: { record, refund, approve, ... },
pos: { createOrder, voidOrder, settleOrder, ... },
inventory: { getItems, addPurchase, adjustStock, ... },
reports: { getOccupancy, getRevenue, getProfitLoss, ... },
auth: { login, logout, validateSession },
app: { checkForUpdates, getVersion, setTestOfflineMode },
import: { parseExcel, checkDuplicates, execute, undoBatch },
data: { exportAll, saveExcel },
email: { sendInvoice, sendConfirmation, sendQuotation },
push: { subscribe, unsubscribe, sendBroadcast },
// ... 15+ namespaces total
}
```

# 3. Technology Stack

## 3.1 Frontend (Desktop Renderer)

| Library / Tool | Version | Role |
|---|---|---|
| React | 18.2 | Component model, rendering |
| react-router-dom | 6.22 | Client-side routing (20+ routes) |
| TailwindCSS | 3.4 | Utility-first styling, dark mode |
| lucide-react | 0.344 | SVG icon set (40+ icons used) |
| date-fns | 3.3 | Date parsing, formatting, ranges |
| xlsx | 0.18 | Excel import parsing and export generation |
| Vite | 5.1 | Build tool, HMR, code splitting |
| electron-vite | 2.0 | Electron + Vite integration |

## 3.2 Desktop (Main Process)

| Library / Tool | Version | Role |
|---|---|---|
| Electron | 29 | Native desktop shell (Windows) |
| better-sqlite3 | 12.2 | Local offline SQLite database |
| @supabase/supabase-js | 2.39+ | Supabase client (RPC, auth, storage) |
| bcryptjs | 2.4 | Password hashing (cost=10) |
| nodemailer | 6.10 | SMTP email sending |
| electron-updater | 6.8 | GitHub Releases auto-update |
| electron-builder | 24.13 | NSIS installer generation |

## 3.3 Manager PWA

| Library / Tool | Version | Role |
|---|---|---|
| React | 19 | UI framework |
| Vite | 5 | Build tool |
| TailwindCSS | 3.4 | Styling |
| @supabase/supabase-js | 2.39+ | Direct database access |
| Service Worker | Browser native | Offline cache, push notifications |

| | | |
|---|---|---|
| Netlify | Hosting | CI/CD deployment |

## 3.4 Backend Infrastructure

| Service | Provider | Purpose |
|---|---|---|
| Relational database | Supabase / PostgreSQL 15 | All persistent data |
| RPC functions | PL/pgSQL (50+ functions) | Server-side business logic |
| Row-Level Security | PostgreSQL RLS policies | Multi-tenant data isolation |
| Edge functions | Supabase Deno runtime | Push notifications, email confirmation |
| File storage | Supabase Storage | Guest ID photos, lodge logos |
| Session management | app_sessions table | Desktop (7d) + PWA (12h) tokens |

# 4. Database Design

The Supabase PostgreSQL database is the single source of truth. Every entity is scoped to a **lodge_id** and protected by Row-Level Security policies. The schema follows a delta-based financial model: balances are never stored directly; they are always computed from immutable transaction records.

## 4.1 Core Entity Tables

### bookings

```
id uuid PRIMARY KEY
lodge_id uuid REFERENCES lodges
customer_id uuid REFERENCES customers
room_id uuid REFERENCES rooms
check_in date
check_out date
adults int
children int
total_amount numeric(12,2) -- base rate + booking_charges
amount_paid numeric(12,2) -- SUM(payments.amount) — never written directly
payment_status text -- 'unpaid' | 'partial' | 'paid' (derived server-side)
status text -- 'confirmed' | 'checked_in' | 'checked_out' | 'cancelled'
invoice_number text UNIQUE per lodge
deposit_amount numeric(12,2) -- requested deposit at booking creation
payment_method text
create_idempotency_key text UNIQUE -- prevents duplicate offline sync
created_at timestamptz
updated_at timestamptz
created_by uuid REFERENCES users
```

### payments

This is the immutable financial ledger. Each row is a delta — never a running total.

```
id uuid PRIMARY KEY
booking_id uuid REFERENCES bookings
lodge_id uuid
amount numeric(12,2) -- positive = payment, negative = refund
method text -- 'cash' | 'card' | 'mobile' | 'cheque' | ...
type text -- 'payment' | 'refund'
paid_at timestamptz
recorded_by uuid REFERENCES users
notes text
idempotency_key text UNIQUE -- prevents duplicate payment on sync retry
```

### rooms

```
id uuid PRIMARY KEY
lodge_id uuid
room_number text
room_type text
rate_per_night numeric(10,2)
max_occupancy int
status text -- 'available' | 'unavailable' | 'maintenance'
amenities jsonb[]
photos text[] -- Supabase Storage URLs
housekeeping_status text
housekeeping_notes text
```

### pos_orders

```
id uuid PRIMARY KEY
lodge_id uuid
outlet_id uuid REFERENCES outlets
order_number text UNIQUE per lodge
total_amount numeric(10,2)
paid_amount numeric(10,2)
status text -- 'open' | 'settled' | 'voided'
customer_type text -- 'room' | 'walkin'
room_id uuid NULLABLE -- set when customer_type = 'room'
booking_folio_id uuid NULLABLE -- links order to booking folio
created_at timestamptz
created_by uuid REFERENCES users
```

## 4.2 Supporting Tables

| Table | Purpose | Key Fields |
|---|---|---|
| customers | Guest profiles and blacklist tracking | name, email, phone, id_number, is_blacklisted |
| booking_charges | Ad-hoc charges added to booking total | booking_id, description, amount, status |
| invoices | Invoice register per booking | booking_id, invoice_number, issued_at |
| quotations | Pre-booking price quotes | Same structure as bookings; status='draft'\|'sent'\|'expired' |
| expenses | Lodge operational costs | category, amount, date, description, recorded_by |
| maintenance_tickets | Room issue tracking | room_id, title, severity, status, assigned_to |
| conference_bookings | Event/meeting room sales | event_name, start_time, end_time, pax, rate |

| | | |
|---|---|---|
| pool_day_use | Day-visit / pool sales | guest_name, pax, amount_paid, date |
| inventory_items | Central stock catalogue | name, category, unit, current_stock, reorder_level |
| inventory_purchases | Stock purchase history | item_id, quantity_purchased, total_cost |
| room_supply_items | Consumable items (towels etc.) | name, unit, central_stock |
| room_supply_movements | Per-room load/use/return log | item_id, room_id, delta, type |
| supply_stocktakes | Periodic stock verification records | item_id, counted_qty, variance |
| lodge_features | Per-lodge feature flag overrides | lodge_id, feature_key, enabled |
| app_sessions | Auth tokens for desktop + PWA | token_hash, session_type, expires_at |
| push_subscriptions | Browser push subscription objects | user_id, lodge_id, subscription_json |
| broadcasts | Admin messages to PWA users | title, body, active_from, active_until, target_roles |
| import_batches | Audit trail for bulk imports | batch_id, filename, row_count, imported_by |
| settings | Lodge configuration (email, branding) | lodge_name, currency, logo_url, email config |
| users | Staff accounts | email, password_hash, role, pwa_enabled, lodge_id |

## 4.3 Key Design Principles

• **Delta-based financials:** amount_paid on bookings is always computed as SUM(payments.amount). Refunds are negative payment entries. No balance is ever stored directly — all balances are derived.

• **Idempotency everywhere:** Every create/payment/refund carries an idempotency_key unique constraint. Duplicate rows from offline sync retries are silently ignored.

• **Immutable ledger:** Payment records are never updated or deleted. Corrections are made via new compensating entries (refund rows).

• **Server-enforced state transitions:** Payment status ('unpaid' → 'partial' → 'paid') is computed inside RPC functions, never by the frontend.

• **Lodge scoping:** Every table has a lodge_id column. RLS policies call app_lodge_access(lodge_id) to enforce that users can only see their own lodge's data.

# 5. Supabase RPC Functions

All mutations in Boroko Bookings flow through **SECURITY DEFINER PostgreSQL functions**. This is the cornerstone of the system's financial integrity: the database enforces business rules at the server level, making it impossible for a malicious or buggy frontend to corrupt financial state.

> **Rule: No INSERT/UPDATE to financial tables directly from the client. All mutations must call an RPC function. This applies equally to desktop and PWA.**

## 5.1 Booking RPC Functions

| Function | Key Parameters | Returns | Guarantee |
|---|---|---|---|
| create_booking | lodge_id, customer_id, room_id, check_in, check_out, total_amount, idempotency_key | jsonb {success, booking_id, idempotent} | Room overlap check; atomic booking+invoice+payment |
| update_booking | booking_id, lodge_id, fields... | jsonb {success} | Validates status transition; records updater |
| cancel_booking | booking_id, lodge_id, reason | jsonb {success} | Sets status=cancelled; triggers email if configured |
| check_in_booking | booking_id, lodge_id | jsonb {success} | Status must be confirmed; sets checked_in |
| check_out_booking | booking_id, lodge_id | jsonb {success} | Validates outstanding balance; sets checked_out |
| add_booking_charge | booking_id, lodge_id, description, amount | jsonb {success, charge_id} | Adds to total_amount atomically |
| delete_booking_charge | charge_id, lodge_id | jsonb {success} | Sets status=deleted; reduces total_amount |

## 5.2 Payment RPC Functions

```
-- Record a payment (positive delta)
update_booking_payment(
p_booking_id uuid,
p_lodge_id uuid,
p_amount numeric, -- must be > 0
p_method text,
p_type text, -- 'payment'
p_idempotency_key text -- UUID, unique per payment attempt
) RETURNS jsonb
-- Request a refund (with optional cancellation fee retention)
```

```
record_booking_refund(
p_booking_id uuid,
p_lodge_id uuid,
p_retained_percent numeric, -- e.g. 20 = keep 20% as cancellation fee
p_method text,
p_notes text,
p_recorded_by uuid,
p_idempotency_key text
) RETURNS jsonb
-- Finance approval step (two-step refund flow)
approve_booking_refund(
p_booking_id uuid,
p_lodge_id uuid,
p_refund_amount numeric,
p_idempotency_key text
) RETURNS jsonb
```

## 5.3 POS RPC Functions

| Function | Purpose | Access |
|---|---|---|
| create_pos_order | Create order with items; reduces inventory if linked | Cashier+ |
| settle_pos_order | Mark order as settled; record payment method | Cashier+ |
| void_pos_order | Void open order; requires supervisor permission | Supervisor+ |
| approve_void_with_pin | Supervisor PIN approval for void | Supervisor+ |
| add_order_item | Add item to open order | Cashier+ |
| remove_order_item | Remove item from open order | Cashier+ |
| apply_order_discount | Apply % or fixed discount to order | Supervisor+ |

## 5.4 Inventory RPC Functions

| Function | Purpose |
|---|---|
| create_inventory_item | Add new stock item to catalogue |
| update_inventory_item | Edit name, category, unit, reorder level |
| delete_inventory_item | Soft-delete item (marks inactive) |
| add_inventory_purchase | Record stock purchase; increases current_stock |
| adjust_inventory_stock | Manual adjustment (delta); logs reason |
| load_supply_to_room | Move supply items from central stock to a room |

| | |
|---|---|
| use_supply_in_room | Record consumption of supply in a room |
| return_supply_from_room | Return unused supply back to central stock |

## 5.5 Reporting & Dashboard RPC Functions

| Function | Returns |
|---|---|
| get_dashboard_snapshot | Occupancy, today's revenue, unpaid count — fast aggregation |
| get_occupancy_report | Day-by-day occupancy rate and room breakdown |
| get_revenue_report | Revenue by category (bookings, POS, day-use) and payment method |
| get_profit_loss | Income vs expenses vs COGS; net profit per period |
| get_outlet_profit_loss | Per-outlet POS performance with item-level margin |
| get_room_profitability | Revenue, occupancy, ADR per room type |
| get_financial_audit | Strict or loose mode booking financials for audit |

## 5.6 Authentication RPC Functions

```
-- Validate credentials; return session token
authenticate_manager(
p_identifier text, -- email or username
p_password text,
p_lodge_id uuid DEFAULT NULL -- required if multi-lodge user
) RETURNS jsonb {success, user, token, lodges}
-- Validate session on every PWA request
validate_manager_session(
p_session_token text
) RETURNS TABLE (user_id, lodge_id, role, expires_at, ...)
-- Revoke session on logout
logout_manager_session(
p_session_token text
) RETURNS void
```

# 6. Security Architecture

## 6.1 Row-Level Security (RLS)

Every table that holds lodge-specific data is protected by a PostgreSQL RLS policy that calls the app_lodge_access(lodge_id) function. This function reads the current session's lodge_id from the request context set by the session validation middleware.

```
-- Example: bookings table RLS policy
CREATE POLICY bookings_lodge_scope ON bookings
FOR ALL
USING (public.app_lodge_access(lodge_id));

-- The helper function
CREATE OR REPLACE FUNCTION public.app_lodge_access(target_lodge_id uuid)
RETURNS boolean AS $$
SELECT current_setting('app.lodge_id', true)::uuid = target_lodge_id
OR current_setting('app.is_master_admin', true)::boolean = true;
$$ LANGUAGE sql STABLE SECURITY DEFINER;
```

The session token passed in the x-boroko-session header is validated by an Edge Function or RPC call that sets the app.lodge_id and app.role session-level settings before any query runs.

## 6.2 Role-Based Access Control (RBAC)

| Role | Primary Scope | POS | Finance | Reports | Staff Mgmt | PWA |
|---|---|---|---|---|---|---|
| Cashier | Single POS outlet | Yes (own outlet) | No | No | No | No |
| Supervisor | Single POS outlet | Yes + voids | No | POS only | No | No |
| Receptionist | Bookings + Guests | No | Payments only | No | No | No |
| Operations | Rooms + Housekeeping | No | No | No | No | No |
| Finance | Payments + Expenses | View only | Full | Full | No | No |
| Manager | Full lodge | Full | Full | Full | View | Yes |
| Admin | Full lodge + Config | Full | Full | Full | Full | Yes |
| Super Admin | Cross-lodge + Billing | Full | Full | Full | Full | Yes |

## 6.3 Granular Capabilities (80+ permissions)

Access is enforced at the capability level, not just the role level. Key examples:

```
// accessControl.js — capability matrix
'bookings.manage' // Create, edit, cancel bookings
'bookings.check_in_out' // Perform check-in and check-out
'payments.record' // Record payment against a booking
'payments.refund' // Initiate a refund
'payments.approve_refund' // Finance approval of refund (two-step)
'pos.void' // Void a POS order
'pos.discount' // Apply discounts to orders
'pos.price_override' // Override menu item price
'pos.menu_manage' // Add/edit/delete menu items
'inventory.manage' // Full inventory control
'staff.permissions' // Modify other users' capabilities
'reports.full' // Access all financial reports
'audit.close' // Perform and close night audit
'settings.manage' // Change lodge configuration
```

## 6.4 Password Security

- **Desktop/PWA passwords:** bcrypt with cost factor 10 — ~100ms hash time to resist brute force.
- **PWA passwords:** Stored in a separate field (pwa_password_hash) — managers can have a different PIN for PWA access.
- **Session tokens:** SHA-256 hashed before storage in app_sessions — raw token never persists.
- **SMTP passwords:** Stored encrypted in the settings table — never returned to frontend in plaintext.
- **Payment data:** Only amounts are stored, never card numbers or banking details.

# 7. Authentication System

## 7.1 Desktop Authentication Flow

```
1. User opens app → AuthContext checks local SQLite for cached session
2. If cached session is valid (< 7 days) → auto-login without network
3. If no cache → show login form
4. User submits email + password:
a. database.js calls authenticate_manager() RPC
b. RPC: bcrypt.compare(password, users.password_hash)
c. RPC: INSERT INTO app_sessions (token_hash, user_id, lodge_id, expires_at)
d. Returns: {success, user, token}
5. Token stored in SQLite + memory
6. All subsequent Supabase calls include header: x-boroko-session: <token>
7. Session expires after 7 days — re-login required
```

## 7.2 PWA Authentication Flow

```
1. User opens PWA in browser → AuthContext checks localStorage for token
2. If token found: validate_manager_session(token) called on load
3. If invalid/expired → redirect to /login
4. Login form: email + password → authenticate_manager() RPC
5. Token stored in localStorage (NOT a cookie — service worker aware)
6. Session TTL: 12 hours (shorter than desktop for security)
7. PWA access limited to users with pwa_enabled = true AND role >= manager
```

## 7.3 Offline Session Handling

When the device is offline, the desktop app validates sessions locally against the SQLite cache. The cached session record includes an offline_lease_expires field set to 7 days from the last successful online validation. This means staff can continue working offline for up to 7 days before being forced to re-authenticate online.

| | |
|---|---|
| **Desktop session TTL** | 7 days (online) + 7 day offline lease |
| **PWA session TTL** | 12 hours (online only — no offline auth) |
| **Token format** | Cryptographically random UUID → SHA-256 before storage |
| **Multi-lodge handling** | authenticate_manager returns list of lodges; user selects one |
| **Session revocation** | logout_manager_session() sets revoked_at; subsequent validations fail |

# 8. Booking & Payment Engine

## 8.1 Booking Lifecycle

```
DRAFT / QUOTATION
■ (convert quotation)
▼
CONFIRMED ■■■■ (initial state for direct bookings)
■ check_in_booking()
▼
CHECKED_IN
■ check_out_booking() ← validates no outstanding balance
▼
CHECKED_OUT

CONFIRMED / CHECKED_IN
■ cancel_booking()
▼
CANCELLED (triggers refund flow if amount_paid > 0)
```

## 8.2 Payment Recording — The Correct Pattern

Every payment, regardless of amount, must flow through the **update_booking_payment** RPC. Direct inserts to the payments table are blocked by RLS. This ensures payment_status is always correctly derived and audit trails are complete.

```
// CORRECT — Always use RPC
const { data, error } = await supabase.rpc('update_booking_payment', {
p_booking_id: bookingId,
p_lodge_id: lodgeId,
p_amount: 5000, // always positive for a payment
p_method: 'cash',
p_type: 'payment',
p_idempotency_key: crypto.randomUUID() // client generates; server deduplicates
});
// WRONG — Never do this
await supabase.from('payments').insert({ booking_id, amount: 5000 }); // BLOCKED by RLS
```

## 8.3 Refund Flow

Refunds use a two-step process to enforce a finance-approval gate. The receptionist initiates the refund, which records the intent but does not release funds. A Finance or Admin user then approves the actual refund amount.

```
// Step 1: Receptionist initiates
```

```
await supabase.rpc('record_booking_refund', {
p_booking_id: bookingId,
p_lodge_id: lodgeId,
p_retained_percent: 20, // 20% cancellation fee — lodge keeps this
p_method: 'mobile_money',
p_notes: 'Guest cancelled due to flight change',
p_recorded_by: userId,
p_idempotency_key: crypto.randomUUID()
});
// Step 2: Finance approves and executes
await supabase.rpc('approve_booking_refund', {
p_booking_id: bookingId,
p_lodge_id: lodgeId,
p_refund_amount: 4000, // 80% of original payment
p_idempotency_key: crypto.randomUUID()
});
```

## 8.4 Room Availability Checking

The **create_booking** RPC performs an overlap check before inserting, preventing double-bookings even when multiple receptionists work simultaneously.

```
-- Inside create_booking RPC (simplified)
IF EXISTS (
SELECT 1 FROM bookings
WHERE room_id = p_room_id
AND lodge_id = p_lodge_id
AND status NOT IN ('cancelled', 'checked_out')
AND check_in < p_check_out
AND check_out > p_check_in
) THEN
RETURN jsonb_build_object('success', false, 'error', 'room_conflict');
END IF;
```

## 8.5 Booking Charges (Folio)

Additional charges (restaurant, minibar, late checkout, etc.) can be added to a booking's folio at any time before checkout. Each charge atomically increases total_amount, which immediately affects the payment_status calculation.

- POS orders linked to a booking automatically create booking_charge records.
- Charges can be soft-deleted (status='deleted') — they reduce total_amount.
- All charges are printed on the final invoice.

# 9. Point of Sale (POS) System

## 9.1 Order Workflow

```
OPEN ORDER
■■ Add items (menu_item_id, quantity, unit_price, discount)
■■ Apply order-level discount (supervisor permission)
■■ Assign to room → creates booking_folio link
■
■■ SETTLE → records payment method → status = 'settled'
■ (inventory depletion committed)
■
■■ VOID → supervisor approval → status = 'voided'
(inventory depletion reversed)
```

## 9.2 Outlet Scoping

Each lodge can have multiple outlets (e.g., Restaurant, Bar, Poolside). Staff are assigned to one or more outlets:

- **Cashier/Supervisor:** Can only process orders for their assigned outlet(s).
- **Manager/Admin:** Can view and manage all outlets.
- **Reports:** Can be filtered by outlet or shown as combined total.
- **Outlet P&L;:** Separate profit/loss statement per outlet showing item-level margin.

## 9.3 Inventory Integration

Menu items can be linked to inventory items. When an order is settled, the POS system reduces the inventory stock by the configured depletion quantity. This enables real-time stock tracking without manual adjustments.

```
-- Menu item schema (relevant fields)
menu_items: {
id, outlet_id, name, price,
inventory_item_id, -- FK to inventory_items (nullable)
depletion_qty -- how much stock is consumed per unit ordered
}
-- When order settled:
UPDATE inventory_items
SET current_stock = current_stock - (item.quantity * menu_item.depletion_qty)
WHERE id = menu_item.inventory_item_id;
```

## 9.4 Room Folio Integration

When a POS order is assigned to a room with an active booking, the system automatically creates a booking_charge entry. This means the guest's folio balance updates in real time and the amount owed at checkout reflects all F&B; spend.

```
// POS order assigned to room
await supabase.rpc('create_pos_order', {
payload: {
outlet_id: outletId,
customer_type: 'room',
room_id: room.id,
booking_folio_id: activeBooking.id, // triggers booking_charge creation
items: [{ menu_item_id, quantity, unit_price }]
}
});
// Result: booking.total_amount += order.total_amount (atomic in RPC)
```

## 9.5 Supervisor Void with PIN

Voiding a settled or open order requires supervisor authorisation. The system supports either a session-based permission check or a PIN-based approval flow where the supervisor enters their 4-digit PIN without logging in as a different user.

# 10. Inventory & Room Supplies

## 10.1 Inventory Module

The inventory module tracks consumable stock for F&B; operations. It supports multiple categories, reorder alerts, purchase history, and manual adjustments.

| | |
|---|---|
| **Categories** | Bar, Kitchen, Other |
| **Units** | bottle, can, piece, kg, L, ml, carton, box |
| **Stock tracking** | current_stock updated on purchase, POS depletion, and manual adjustment |
| **Reorder alerts** | Dashboard shows items where current_stock <= reorder_level |
| **Purchase history** | inventory_purchases table records quantity, cost, and date |
| **Valuation** | latest_unit_cost * current_stock used for COGS in P&L; report |

## 10.2 Room Supplies Module

Room supplies tracks consumables allocated to specific rooms: linens, toiletries, water bottles, and similar items. This module provides an audit trail of every movement between central storage and individual rooms.

```
CENTRAL STOCK (supply_items.central_stock)
■
■ load_supply_to_room(item_id, room_id, qty)
▼
ROOM STOCK (room_supply_stock.quantity per room)
■
■ use_supply_in_room(item_id, room_id, qty) ← records consumption
■ return_supply_from_room(item_id, room_id, qty) ← returns to central
▼
MOVEMENT LOG (room_supply_movements)
Audit trail: who, what, when, delta, type
```

Periodic stocktakes record counted quantities per item, and the system calculates the variance between expected and counted stock to identify losses or theft.

# 11. Reporting & Analytics

## 11.1 Report Types

| Report | Scope | Key Metrics | Export |
|---|---|---|---|
| Dashboard Snapshot | Today | Occupied rooms, day's revenue, unpaid bookings, pending online requests | No |
| Occupancy Report | Date range | Room-nights sold, occupancy %, RevPAR, forecast | Excel + PDF |
| Revenue Report | Date range | Booking revenue, POS revenue, day-use, payment method mix, ADR | Excel |
| Profit & Loss | Date range | Revenue – COGS – Expenses = Net profit; by category | Excel + PDF |
| Outlet P&L; | Date range | Per-outlet revenue, COGS, margin %; item-level breakdown | Excel |
| Room Profitability | Date range | Revenue, nights, ADR, occupancy per room type | Excel |
| Financial Audit | Date range | All transactions; strict mode (confirmed only) or loose mode | Excel |
| Night Audit | Single night | Cash reconciliation, check-ins/outs, variances | PDF |
| Expense Summary | Date range | Expenses by category; vs prior period | Excel |
| Inventory Valuation | Point in time | Stock on hand × unit cost per item | Excel |

## 11.2 Dashboard Snapshot

The dashboard is powered by the **get_dashboard_snapshot** RPC, which runs a series of optimised aggregate queries and returns a single JSON object. This minimises round-trips and keeps the dashboard load time under 500ms.

```
// Snapshot response shape
{
occupied_rooms: 12,
```

```
total_rooms: 20,
occupancy_rate: 0.60,
todays_revenue: 15400.00,
outstanding_balance: 8200.00, // sum of unpaid/partial bookings
unpaid_bookings_count: 4,
overdue_payments_count: 1, // checked_out but balance > 0
pending_online_requests: 2,
low_stock_items: 3, // inventory at or below reorder level
todays_checkouts: 5,
todays_checkins: 7
}
```

## 11.3 Financial Audit Modes

**Strict mode** includes only confirmed/checked-in/checked-out bookings. **Loose mode** includes all statuses including cancelled. This matters because cancelled bookings with retained fees still represent revenue.

# 12. Offline-First Synchronization

## 12.1 Local Storage Architecture

Each lodge has its own SQLite database file stored on disk at a path derived from the lodge UUID. The SQLite database acts as a read-through cache and a sync queue.

```
// Storage path
~/.local/share/boroko-bookings/[lodge-uuid]/local-storage.sqlite

// Cached tables (mirrors of Supabase)
bookings, customers, payments, rooms, users, settings,
outlets, menu_items, quotations, expenses, pos_orders,
pos_order_items, inventory_items, inventory_purchases,
supply_items, supply_movements, supply_allocations, import_batches

// Sync management tables
sync_queue (position, payload_json, updated_at)
sync_failed (position, payload_json, error_message, failed_at)
```

## 12.2 Sync Queue Operation

```
// 1. User performs action offline
const order = { id: newUUID(), outlet_id, items: [...] };
db.writeCache('pos_orders', [...existing, order]); // optimistic UI update
db.queueOperation({
_queue_id: order.id,
_depends_on: null, // no dependency
table: 'create_pos_order', // RPC name
type: 'rpc',
args: { payload: order },
_sync_state: 'pending',
});

// 2. Every 5 minutes: sync worker runs
async function syncWorker() {
if (!navigator.onLine) return;
const pending = db.getPendingQueue();
for (const item of pending) {
const result = await supabase.rpc(item.table, item.args);
if (result.data?.success) {
db.markSynced(item._queue_id);
} else {
db.markFailed(item._queue_id, result.error);
}
}
```

```
}
```

## 12.3 Conflict Resolution Strategies

| Conflict Type | Resolution Strategy |
|---|---|
| Duplicate payment on retry | idempotency_key UNIQUE constraint — DB silently ignores duplicate |
| Duplicate booking on retry | create_idempotency_key UNIQUE — returns {idempotent: true} |
| Room double-booking | RPC overlap check — second booking returns {error: 'room_conflict'} |
| Stale cache read | On reconnect: full re-fetch from Supabase overwrites local cache |
| Failed sync item | Moved to sync_failed table; UI shows manual retry option |
| Dependency chain failure | _depends_on field pauses dependent operations until parent succeeds |

## 12.4 Offline Indicators

The desktop app status bar shows a persistent sync status indicator: green (synced), amber (pending items), red (failed items). Bookings with pending or failed sync state show a warning badge and block void/edit actions until resolved.

# 13. Feature Flags & Subscriptions

## 13.1 Plan Feature Map

```
// database.js
const PLAN_FEATURE_MAP = {
Starter: {
reports: false, expenses: false, staff: false, audit: false,
conference: false, pool: false, import: false,
pos: false, inventory: false, supplies: false, pwa: false
},
Standard: {
reports: true, expenses: true, staff: true, audit: true,
conference: true, pool: true, import: true,
pos: false, inventory: false, supplies: false, pwa: false
},
Pro: {
reports: true, expenses: true, staff: true, audit: true,
conference: true, pool: true, import: true,
pos: true, inventory: true, supplies: true, pwa: true
}
};
```

## 13.2 Feature Enforcement

Feature flags are loaded at login and stored in a React Context. Routes and nav items are conditionally rendered. For extra protection, the Supabase lodge_features table can override any feature per lodge, and the RPC functions validate entitlements server-side before performing restricted operations.

```
// React component — client-side gate
function UpgradeWall({ feature, children }) {
const features = useContext(FeaturesContext);
if (!features[feature]) {
return <UpgradeBanner requiredPlan='Pro' feature={feature} />;
}
return children;
}
// Usage in App.jsx
<UpgradeWall feature='pos'>
<Route path='/pos' element={<POS />} />
</UpgradeWall>
```

## 13.3 Feature Flag Polling

The desktop app polls the feature flags every 60 seconds. If a subscription lapses or a feature is disabled by an admin, the UI adapts within one minute without requiring a restart. The offline cache preserves the last-known subscription state for up to 7 days.

# 14. Manager PWA

## 14.1 Purpose and Audience

The Manager PWA is a mobile-first React application targeting lodge managers and admins who need to monitor operations from a phone or tablet without being tied to a desktop workstation. It is a **read-heavy, lightweight companion** to the full Electron desktop app.

## 14.2 Page Inventory

| Page | Capabilities | Create? | Edit? |
|---|---|---|---|
| Login | Email + password auth; lodge selector for multi-lodge users | N/A | N/A |
| Dashboard | KPI snapshot: occupancy, revenue, outstanding balance | No | No |
| Bookings | View upcoming/past bookings; filter by date, status | No | Payment only |
| Rooms | Room occupancy grid; housekeeping status | No | Housekeeping status |
| Guests | Customer list; view profile, booking history | No | No |
| Quotations | View quotations and status | No | No |
| Invoices | View invoice list; trigger email resend | No | No |
| Expenses | Expense summary by category and date range | View only | No |
| Inventory | Stock levels; reorder alerts | View only | No |
| Reports | All report types; download Excel | N/A | N/A |
| Audit | Night audit view; close audit | No | Yes (close only) |
| Conference | Event bookings list | No | No |
| Day Use | Pool/day-use sales list | No | No |
| Money | Payment tracking dashboard | No | No |
| More | Profile, push notification settings, logout | N/A | N/A |

## 14.3 PWA Architecture

```
manager-pwa/
■■■ src/
■ ■■■ pages/ # One file per route
■ ■■■ lib/
■ ■ ■■■ api.js # All Supabase RPC calls (direct — no database.js)
■ ■ ■■■ supabase.js # Supabase client init with session header injection
■ ■ ■■■ access.js # Capability checks for PWA-specific permissions
■ ■■■ contexts/
■ ■ ■■■ AuthContext.jsx # Session token, user profile, lodge context
■ ■■■ service-worker.js # Offline cache + push handler
■■■ public/
■ ■■■ manifest.json # PWA manifest (name, icons, theme_color)
■■■ vite.config.js # Build config with PWA plugin
```

## 14.4 What the PWA Cannot Do

- Create new bookings (desktop only — requires room conflict check + offline idempotency).
- Convert quotations to bookings.
- Create or edit menu items.
- Manage staff accounts or permissions.
- Access POS terminal (cashier/supervisor functions).
- Import data from Excel.
- Configure lodge settings.

*These limitations are intentional — they reduce the blast radius of mobile access and ensure that destructive or complex operations require a full desktop session.*

# 15. Push Notifications & Broadcasts

## 15.1 Push Notification Infrastructure

Push notifications use the standard Web Push API. The PWA subscribes and stores the browser's push subscription object in Supabase. A Supabase Edge Function (Deno runtime) handles sending notifications using the VAPID protocol.

```
// 1. PWA subscribes
const registration = await navigator.serviceWorker.ready;
const subscription = await registration.pushManager.subscribe({
userVisibleOnly: true,
applicationServerKey: VITE_VAPID_PUBLIC_KEY
});
await supabase.from('push_subscriptions').upsert({
user_id, lodge_id,
subscription: JSON.stringify(subscription)
});
// 2. Server sends notification (Edge Function: send-push)
await supabase.functions.invoke('send-push', {
body: {
lodge_id: lodgeId,
title: 'New Online Booking Request',
body: 'Room 201 — 3 nights — John Doe',
url: '/bookings'
}
});
// 3. Service worker receives and displays
self.addEventListener('push', event => {
const data = event.data.json();
event.waitUntil(
self.registration.showNotification(data.title, {
body: data.body,
icon: '/icon-192.png',
data: { url: data.url }
})
);
});
```

## 15.2 Broadcast System

Admins can push broadcast messages to all connected PWA users. Broadcasts are stored in a Supabase table with active_from and active_until timestamps and an optional target_roles filter. The PWA uses

Supabase Realtime to receive broadcasts instantly without polling.

```
// Admin creates broadcast
await supabase.rpc('create_broadcast', {
title: 'System Maintenance',
body: 'The system will be offline 02:00–03:00 tonight.',
active_from: new Date().toISOString(),
active_until: tomorrowISO,
target_roles: ['manager', 'admin']
});
// PWA subscribes to real-time broadcast changes
const channel = supabase
.channel('broadcasts')
.on('postgres_changes', { event: '*', schema: 'public', table: 'broadcasts' },
() => fetchActiveBroadcasts()
)
.subscribe();
```

# 16. Email Integration

## 16.1 SMTP Configuration

Email is sent from the desktop app's main process using Nodemailer. The admin configures SMTP credentials in Settings, and the app supports preset configurations for the most common providers.

| Provider Preset | Host | Port | Auth |
|---|---|---|---|
| Gmail | smtp.gmail.com | 587 (STARTTLS) | App Password required |
| Outlook / Hotmail | smtp.office365.com | 587 | OAuth2 or App Password |
| Zoho Mail | smtp.zoho.com | 587 | App Password |
| cPanel / Hosting | mail.yourdomain.com | 587 or 465 | Email + password |
| Custom SMTP | Configurable | Configurable | Username + password |

## 16.2 Automated Email Triggers

| Email Type | Trigger | Recipients | Auto-send Setting |
|---|---|---|---|
| Booking Confirmation | Booking status → confirmed | Guest email | auto_send_booking_confirmation |
| Booking Invoice | Invoice issued | Guest email | auto_send_booking_invoice |
| Quotation | Quotation marked as sent | Guest email | auto_send_quotations |
| Cancellation Notice | Booking cancelled | Guest email | auto_send_booking_cancellation |
| Refund Notification | Refund approved | Guest email | auto_send_refund |

## 16.3 Email Templates

All email templates are generated in HTML by the **emailNotifications.js** module. Templates are inline-styled for email client compatibility and include:

- Lodge logo and branding (fetched from settings.logo_url).
- Booking details: guest name, room, dates, adults/children.
- Itemised charge breakdown with subtotals.
- Payment status and outstanding balance.
- A WhatsApp link pre-populated with the lodge phone number for easy guest follow-up.
- Booking FAQ text (configurable in settings).

# 17. Data Import & Export

## 17.1 Bulk Import

Managers can import historical booking data from a standardised Excel template. The import pipeline validates, deduplicates, and inserts records in a single auditable batch that can be reversed (undo import).

```
// Import workflow
1. Admin downloads Excel template from Settings → Data Management
2. Template filled with: guest name, email, phone, room number,
check-in, check-out, total amount, payment method, notes
3. Upload Excel → parseExcel() reads with xlsx library
4. checkDuplicates() compares against existing bookings (same room+dates)
5. Preview shown with conflict flags
6. Execute import:
- Creates customer records (upsert by email)
- Creates booking records via create_booking RPC (idempotency keys)
- Logs import_batches record: batch_id, row_count, imported_by, filename
7. Undo: undoBatch(batchId) deletes all records linked to batch_id
```

## 17.2 Data Export

| Export Type | Format | Contents |
|---|---|---|
| Bookings report | Excel (.xlsx) | All booking details + payment status for date range |
| Full data export | JSON + Excel | All lodge data: bookings, guests, payments, expenses, POS |
| Occupancy report | Excel | Room-by-room occupancy grid |
| P&L; report | Excel + PDF | Formatted profit/loss statement |
| Night audit | PDF | Formatted audit close document |
| Guest history | Excel | All bookings for a specific guest |

# 18. Performance & Build

## 18.1 Code Splitting Strategy

The Electron renderer is built with Vite and uses **manualChunks** to split heavy dependencies into separate bundles. This reduces the initial JS load and speeds up app startup.

```
// electron.vite.config.js — manualChunks
manualChunks: {
'react-vendor': ['react', 'react-dom', 'react-router-dom'],
'icons': ['lucide-react'],
'dates': ['date-fns'],
'supabase': ['@supabase/supabase-js'],
'xlsx': ['xlsx'], // only loaded on import/export pages
}
```

## 18.2 Lazy Loading

```
// Heavy pages are code-split and only loaded when the route is visited
const Reports = lazy(() => import('./components/Reports'));
const DataManagement = lazy(() => import('./components/DataManagement'));
const POS = lazy(() => import('./components/POS'));

<Suspense fallback={<PageLoader />}>
<Routes>
<Route path='/reports' element={<Reports />} />
<Route path='/pos' element={<POS />} />
</Routes>
</Suspense>
```

## 18.3 Release Pipeline

```
// Build targets
npm run build → Compiles renderer + main with electron-vite
npm run dist → Packages as NSIS installer (Windows .exe)

// package.json build config
{
'appId': 'com.boroko.bookings',
'productName': 'Boroko Bookings',
'publish': {
'provider': 'github',
'owner': 'Rabafi',
'repo': 'boroko-bookings-releases'
}
}
```

```
// Auto-update check on startup
autoUpdater.checkForUpdatesAndNotify();
// User prompted to install update on next restart
```

## 18.4 Scheduled Financial Validation

The desktop app runs a background financial validation job every 2 hours. This checks for data integrity issues and alerts the admin if any are found.

- amount_paid on each booking matches SUM of its payments records.
- No booking has amount_paid greater than total_amount (overpayment check).
- payment_status correctly reflects the computed balance.
- No orphaned payment records (payments referencing deleted bookings).
- Alerts written to financial-validation-alerts.json and shown in the Reports module.

# 19. Known Issues & Roadmap

## 19.1 Known Issues (Active)

| Issue | Severity | Impact | Status |
|---|---|---|---|
| Deposit recording inconsistency | High | Deposits at booking creation not always recorded as payments — can cause balance discrepancy | Under investigation |
| database.js single-file size | Medium | 13,000-line file is hard to maintain and test in isolation | Planned modularisation |
| Inventory depletion not atomic with order creation | Medium | Concurrent POS orders on same item could oversell before stock check runs | Planned fix |
| Sync queue no dependency timeout | Low | A failed parent in a dependency chain blocks children indefinitely | Planned — add TTL |
| Audit log incomplete | Low | Not all sensitive operations are logged to a central audit trail | Planned |

## 19.2 Architectural Improvements Planned

- **database.js modularisation:** Split into domain-specific modules (bookings.js, payments.js, pos.js, etc.) for testability.
- **Comprehensive idempotency:** Extend idempotency keys to all sync operations, not just payments and bookings.
- **Pessimistic locking:** Add FOR UPDATE locks in RPCs that risk concurrent edit conflicts (e.g., booking checkout).
- **Rate limiting:** Add per-session rate limits on financial RPCs to prevent abuse.
- **Central audit log:** A dedicated audit_log table recording all mutations with before/after state.
- **PWA write capabilities:** Allow managers to create bookings and quotations from the PWA with the same safety guarantees.

## 19.3 Feature Roadmap (Planned)

- **Online booking engine:** Public-facing booking page integrated with the pending request queue.
- **Channel manager integration:** OTA (Airbnb, Booking.com) availability sync.
- **Dynamic pricing:** Automatic rate adjustments based on occupancy thresholds.
- **Customer loyalty module:** Points accumulation and redemption tracking.

- **Mobile POS:** PWA-based cashier terminal for tablet use in F&B; outlets.
- **Multi-currency support:** Prices in local currency with exchange rate conversion for reporting.

# 20. Conclusion

Boroko Bookings is a mature, production-grade hospitality management system that makes deliberate architectural trade-offs to serve its target market: properties operating in environments where connectivity is unreliable, staff turnover is high, and financial accuracy is non-negotiable.

## 20.1 Core Architectural Principles

### 1. All financial mutations are server-enforced.

SECURITY DEFINER RPC functions are the only path to modifying payments, bookings, and charges. No amount of frontend tampering or API miscall can bypass this. The business logic lives in the database, not the client.

### 2. Offline-first is a first-class requirement, not an afterthought.

SQLite caching, sync queues, idempotency keys, and offline session leases are baked into the core architecture. The system degrades gracefully to fully offline mode and resumes without data loss when connectivity returns.

### 3. Multi-tenancy through data, not deployment.

Every table is lodge-scoped and every RLS policy enforces this. One Supabase instance serves all lodges safely. A Super Admin can cross lodge boundaries; no other role can.

### 4. Role-based access as a product feature.

The 80+ capability system lets lodges configure exactly what each staff member can do. This is not just a security feature — it reduces training time and prevents accidental mistakes by junior staff.

### 5. The PWA is a window, not a door.

The Manager PWA deliberately limits write operations. Managers get full visibility into lodge operations from any device, but destructive or complex operations are reserved for the desktop environment where full context, offline safety, and access control are guaranteed.

## 20.2 Final Notes for Developers

> **Golden Rule: If you are adding a mutation (create, update, delete) to ANY financial entity, it MUST go through a Supabase RPC function. It MUST include an idempotency key. It MUST be replicated in both the desktop (database.js) AND the PWA (lib/api.js) if it is needed in both contexts. There are no exceptions to this rule.**

The codebase rewards developers who understand the separation between the Electron main process (database.js) and the PWA's direct API layer. These two paths must always converge on the same RPC functions. Any divergence creates a category of bugs that is very hard to detect in testing and very damaging in production.

*Boroko Bookings v1.2.6 — Technical Architecture Document*
*Generated: 15 April 2026 • Confidential*